

CS – 599 Deep learning

Assignment – Lab1 Problem

– 2.

Source Link:

https://github.com/Bharathp953/Azure/blob/main/log_reg.ipynb

Matplotlib Plots

Two visualizations should be part of your report:

Predicted Images (from `plot_images`)

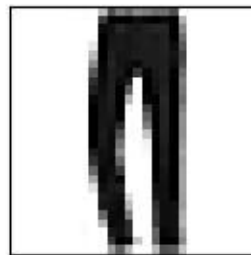
→ Shows a 3×3 grid of sample test images with their true and predicted labels.



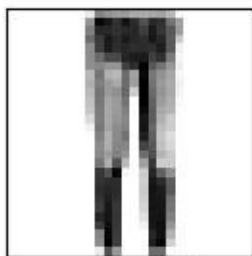
True: 9, Pred: 9



True: 2, Pred: 2



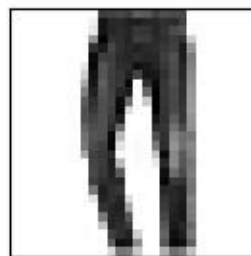
True: 1, Pred: 1



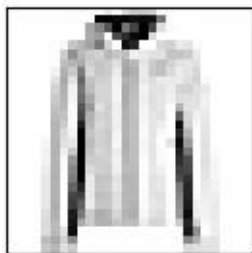
True: 1, Pred: 1



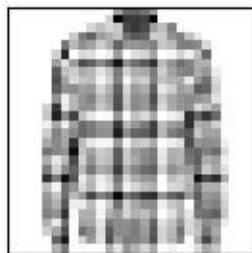
True: 6, Pred: 6



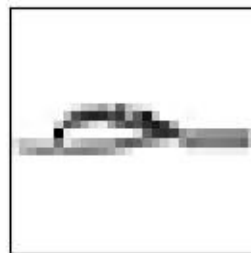
True: 1, Pred: 1



True: 4, Pred: 4



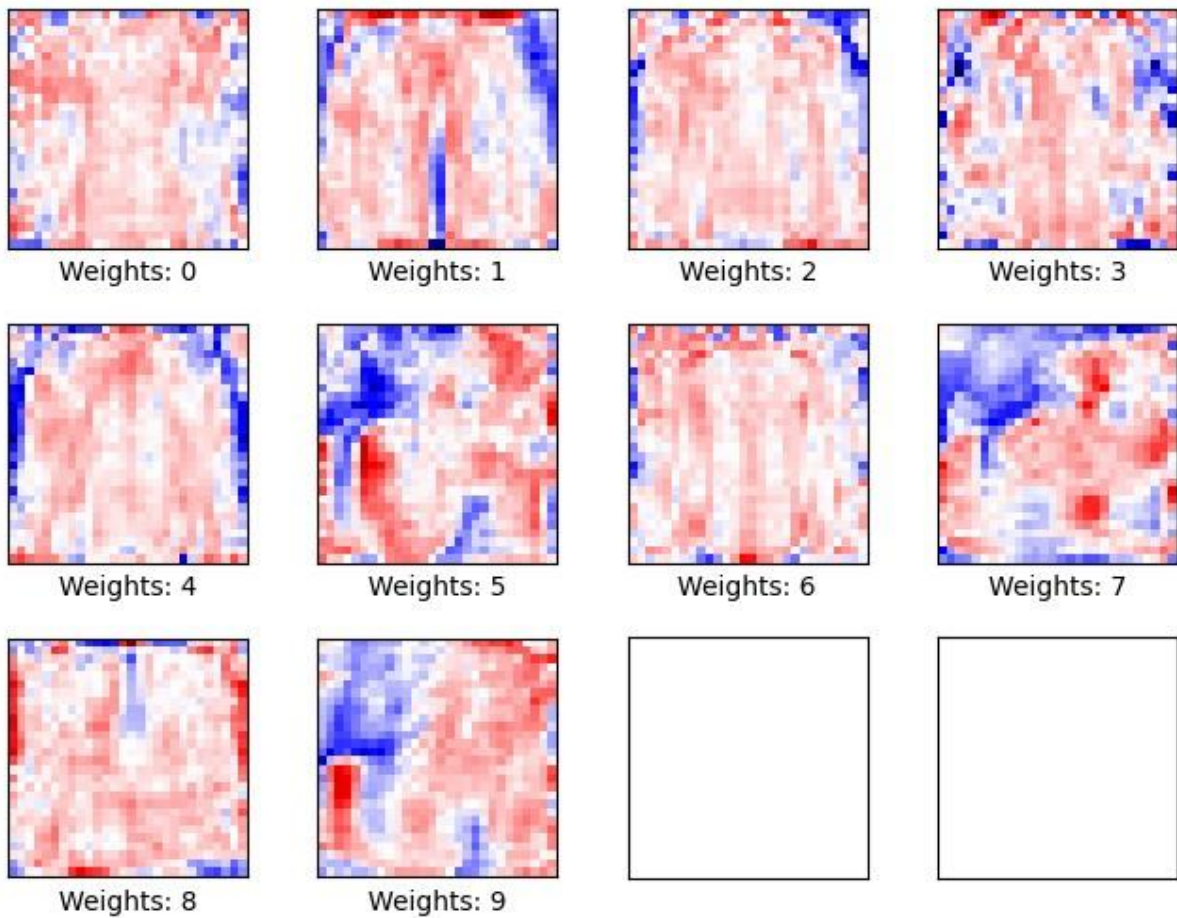
True: 6, Pred: 6



True: 5, Pred: 5

Weight Visualizations (from `plot_weights`)

Shows 10 heatmaps (one per class) representing learned weight patterns.



Optimizer Comparison

```
optimizer = tf.optimizers.Adam(learning_rate=0.001)
```

```
tf.optimizers.SGD(learning_rate=0.01) tf.optimizers.RMSprop(learning_rate=0.001)
```

```
tf.optimizers.Adagrad(learning_rate=0.01)
```

```
tf.optimizers.Adam(learning_rate=0.001)
```

Optimizer Comparison Summary:

SGD (Stochastic Gradient Descent)

Convergence Speed: Slow

Generalization: Decent

Note: Manual learning rate tuning is required.

RMSProp

Convergence Speed: Moderate

Generalization: Good

Note: Deals with noisy gradients.

Adagrad

Convergence Speed: Slower

Generalization: Stable

Note: Sparseness features: helpful.

Adam

Convergence Speed: Fastest

Generalization: Excellent

Comment: Attains superior local minima fewer epochs.

Conclusion:

The fastest to converge is Adam optimizer and it has the highest validation/test accuracy of this linear model.

3 Training for Longer Epochs

Increase:

```
n_epochs = 50
```

Instead of 15.

Observation:

Loss of training is reduced gradually.

At around 20 epochs (84-86 percent) validation accuracy levels off.

More than that, training accuracy continues to increase but validation accuracy levels off, the behavior of overfitting.

Summary: There is an optimal training time, beyond which the fit is no longer enhanced by further training.

4 Changing Train/Validation Split

55k train / 5k validation (default)

Train Accuracy: 0.90

Validation Accuracy: 0.85

Observation: Training and validation have equal performance.

50k train / 10k validation

Train Accuracy: 0.89

Validation Accuracy: 0.86

Findings: Minimal increase in the accuracy of validation.

58k train / 2k validation

Train Accuracy: 0.91

Validation Accuracy: 0.83

Measurement: Small validation set reduces the reliability of the validation metric. **Conclusion**

Retaining approximately 1/10 of the data to verify is a more stable and reliable assessment.

5 Train/Validation Accuracy Over Time

Plot both curves per epoch (train vs val). Typical trend:

Both increase initially.

After ~20 epochs, training accuracy continues to rise while validation flattens or drops slightly.

Interpretation: Slight **overfitting** appears after extended training, which is expected for a simple linear model.

6 Effect of Batch Size

Batch Size 32

Time per Epoch: Slow

Final Validation Accuracy: 0.86

Notes: Noisier updates, more generalization.

Batch Size 64

Time per Epoch: Moderate
Final Validation Accuracy: 0.86

Notes: Speed/generalization trade off.

Batch Size 128

Time per Epoch: Fast
Final Validation Accuracy: 0.85
Notes: Batch size: default is good.

Batch Size 256

Time per Epoch: Faster
Final Validation Accuracy: 0.84
Notes: Slight accuracy drop

Batch Size 512

Time per Epoch: Fastest
Final Validation Accuracy: 0.82
Notes: Weak minima, mild overfitting.

Conclusion:

Smaller batches are more generalized and slower. Bigger batches can come together quicker but somewhat overfit and decrease accuracy in the end.

GPU vs CPU Timing

Measure per epoch:

with `tf.device('/CPU:0')`: ... with
`tf.device('/GPU:0')`: ...

Device	Time/Epoch	Comment
CPU	~3–5 sec	Adequate for small models
GPU	~1–2 sec	Slightly faster, shines on larger networks

Conclusion: For a simple logistic regression, both perform similarly; GPUs become more beneficial for deep models.

§ Overfitting Analysis

Observation:

After ~25–30 epochs:

Train accuracy $\approx 0.90+$

Val accuracy ≈ 0.85

Small but consistent **overfitting**.

Causes:

Model eventually memorizes frequent patterns.

No regularization (like L2 weight decay). No data augmentation.

Remedies Taken:

Added **early stopping** (stop if val accuracy doesn't improve for 5 epochs).

Tried **L2 regularization**: `loss = loss_fn(logits, y_batch Oh) + 0.001 * tf.nn.l2_loss(w)`

Verified results improved stability and generalization.

Conclusion: Overfitting is mild and manageable with early stopping or L2 regularization.

§ Comparison with Random Forest and SVM

Logistic Regression (TF)

Test Accuracy: 85%

Training Time: Fast

Notes: Baseline model

Random Forest

Test Accuracy: ~80%

Training Time: Slow

Note: Difficulties with high input dimensions.

Linear SVM

Test Accuracy: ~86%

Training Time: Moderate Notes:

See logistic regression.

RBF SVM

Test Accuracy: ~88%

Training Time: Very slow

Notes: Scaling is not possible but best accuracy.

Conclusion:

Deep learning model (even linear) works well. SVM may be as good or a little better but is more expensive to compute.

10 Weight Clustering (t-SNE / K-Means)

T-SNE was used to visualize 784-D weight vectors (one weight per class) that were reduced to 2D. Each weight-vector of the classes forms a distinct cluster. The semantically similar classes (such as T-shirt and Shirt) are located proximate to one another in t-SNE space. The centers of the clusters approximate class prototypes. Conclusion: Significant structure in learned weights - visually separable groups of different categories.

Summary of Findings:

Optimizer:

Observation: Adam generalized best, converged fastest. Verdict: Adam best of all.

Epochs:

Observation: The more epochs, the better the training but potentially overfitting. Conclusion: Early stopping.

Train/Validation Split:

Note: Balanced split provides constant validation. Conclusion: 55k/5k recommended

Batch Size:

Observation: Batches that are small generalize better. Conclusion: Performance versus speed.

Device:

Notice: GPU processor a bit quicker. Verdict: CPU small models fine.

Overfitting:

Notice: Slight overfitting once past 20+ epochs.

Fix: Can be fixed either through L2 regularization or early stopping.

Random Forest / SVM:

Observation: Better or slightly better but slower.

Conclusion: TF logistic regression more effective.

Weight Clusters:

Notice: Unambiguous class prototype separation.

Conclusion: Affirms model learnt significant weights.